AI/ML Engineer Assignment

Instructions

Centrifuge machines rely on high-speed rotating components, including bearings, to operate. When bearings degrade, the vibration signature of the machine changes. These changes can manifest in both the time domain (e.g., overall vibration amplitude) and the frequency domain (e.g., emergence of new spectral peaks or shifts in energy distribution across frequency bands).

The provided utility functions convert raw acceleration data into standard vibration analysis domains — acceleration in g, velocity in mm/s, and their respective frequency spectra. You are encouraged to research vibration-based condition monitoring techniques to inform your analysis.

This assignment assesses your ability to work with industrial machinery sensor data, a core part of this role. You will have 48 hours from the time of receiving this assignment to submit your deliverables.

Deliverables

Submit a single ZIP file named `<CandidateName>_AIMLEngineer_Groundup.zip` containing:

1. A PDF report with your answers to all 3 questions
2. A folder containing all source code, including the Jupyter notebook for Question 2

Evaluation Criteria: Technical depth (40%), Code quality (25%), Communication & presentation (20%), Architecture reasoning (15%).

- Please ensure your deliverables are clear, concise, and well-organized.

Dataset

The data provided comes from a sensor attached to a Centrifuge Machine, this sensor measures Vibration(as Acceleration in mm/s², Temperature and Audio data as well as other metadata.



Centrifuge Machine, Sensor is labelled in the red box within a metal housing

Dataset Download Link:

Download: https://drive.google.com/file/d/1gQFwz2n2L9yNq_kB1qfsDSvt0BW68hXb/view?usp=sharing

Channel to Direction Mapping

Sensor Direction	Machine Direction
X	Axial
Y	Horizontal
Z	Vertical

Dataset Columns

Name	Type	Sampli Rate
date	datetime	
Motor DE_Z Short Wave High Frequency	Z Long Wave - High Frequency	51200
Motor DE_Z Long Wave High Frequency	Z Short Wave - High Frequency	51200
Motor DE_X Short Wave Low Frequency	X Short Wave - Low Frequency	4001
Motor DE_Y Short Wave Low Frequency	Y Short Wave - Low Frequency	4001
Motor DE_Z Short Wave High Frequency	Y Short Wave - Low Frequency	5120
Motor DE_temperature	temperature	
Motor DE_Audio Signal	audio	11025
Motor DE_Z Short Wave Low Frequency RMS		
Motor DE_X Short Wave Low Frequency RMS		
Motor DE_Y Short Wave Low Frequency RMS		
Motor DE_Z Speed Total Value		
Motor DE_X Speed Total Value		
Motor DE_Y Speed Total Value		
Motor DE_Z Short Wave High Frequency RMS		
Motor DE_Audio Decibels		
Motor DE_Magnetic Leakage RMS		

Labels

Labelled data (provided to you): 1 Jan 2025 to 4 Jan 2025 : Bearing Fault

Held-out evaluation period (not provided to you): 29 April 2025 – 9 June 2025: Bearing Fault

Your model will be evaluated against data from the held-out period.

Utility Functions

```python
import numpy as np
from scipy import signal

# Unit / domain conversions
def convert_ms2_to_mms2(data: np.ndarray) -> np.ndarray:
    return data * 1000.0


def to_g(data: np.ndarray) -> np.ndarray:
    return data / 9.80665


def integrate_data(data: np.ndarray, sampling_rate: float) ->
np.ndarray:
    return np.cumsum(data) / sampling_rate

# Filters
def bandpass_filter(
    data: np.ndarray,
    low_cutoff: int,
    high_cutoff: int,
    sampling_rate: int,
) -> np.ndarray:
    sos = signal.butter(
        6,
        [low_cutoff, high_cutoff],
        btype="bandpass",
        fs=sampling_rate,
        output="sos",
    )
    return np.asarray(signal.sosfilt(sos, data))


def highpass_filter(
    data: np.ndarray,
    low_cutoff: int,
    sampling_rate: int,
) -> np.ndarray:
    sos = signal.butter(6, low_cutoff, btype="highpass",
fs=sampling_rate, output="sos")
    return np.asarray(signal.sosfilt(sos, data))


# helpers
def remove_settling_time(
    data: np.ndarray,
```

```
        low_cutoff: int,
        original_length: int,
        sampling_rate: int,
    ) -> np.ndarray:
        fmax = sampling_rate / 2.56
        duration = (original_length / 2) / fmax
        multiplier = 2 if duration >= 2 else 1
        start = int(
            multiplier / low_cutoff * sampling_rate,
        )
        start = min(start, len(data) // 10)
        return data[start:]


    def get_rms(data: np.ndarray) -> float:
        return float(np.sqrt(np.mean(np.square(data))))


    def remove_dc_offset(data: np.ndarray) -> np.ndarray:
        return data - np.mean(data)


    def preprocess_acceleration(
        data: np.ndarray,
        sampling_rate: int,
    ) -> np.ndarray:
        length = len(data)
        low_cutoff = 10
        x = to_g(data)
        x = highpass_filter(x, low_cutoff, sampling_rate)
        x = signal.detrend(x, type="linear")
        x = remove_settling_time(x, low_cutoff, length, sampling_rate)
        x = remove_dc_offset(x)
        return x


    def preprocess_velocity(
        data: np.ndarray,
        sampling_rate: int,
    ) -> np.ndarray:
        length = len(data)
        low_cutoff = 10
        high_cutoff = 1000

        x = convert_ms2_to_mms2(data)
        x = integrate_data(x, sampling_rate)
        x = bandpass_filter(x, low_cutoff, high_cutoff, sampling_rate)
        x = signal.detrend(x, type="linear")
        x = remove_settling_time(x, low_cutoff, length, sampling_rate)
        x = remove_dc_offset(x)
        return x
```

```
    def get_fft(
        data: np.ndarray,
        sampling_rate: int,
    ):
        N = len(data)
        line_of_resolution = N / 2.56
        maximum_frequency = sampling_rate / 2.56
        resolution = maximum_frequency / line_of_resolution

        nperseg = int(sampling_rate / resolution)
        nperseg = min(nperseg, len(data))
        noverlap = int(0.5 * nperseg)

        frequencies, psd = signal.welch(
            data,
            fs=sampling_rate,
            window="hann",
            nperseg=nperseg,
            noverlap=noverlap,
            scaling="density",
            detrend=False,
        )
        mask = frequencies <= line_of_resolution
        frequencies = frequencies[mask]
        psd = psd[mask]

        df = frequencies[1] - frequencies[0]
        rms = np.sqrt(psd * df)
        return frequencies, rms

    def get_features(
        data: np.ndarray,
        sampling_rate: int,
    ) -> dict:
        features = {}
        preprocessed_acceleration = preprocess_acceleration(data,
sampling_rate)
        features["acceleration_rms"] = get_rms(preprocessed_acceleration)
        features["acceleration_fft"] = get_fft(preprocessed_acceleration,
sampling_rate)

        preprocessed_velocity = preprocess_velocity(data, sampling_rate)
        features["velocity_rms"] = get_rms(preprocessed_velocity)
        features["velocity_fft"] = get_fft(preprocessed_velocity,
sampling_rate)
        return features
```

Question 1: Exploratory Data Analysis

This section assesses your ability to make sense of machinery sensor data and identify meaningful patterns.

Using the full dataset provided, produce a report that includes:

- Your step-by-step approach to exploring the data
- Key patterns, trends, and anomalies you identified (with supporting visualizations)
- Your hypotheses about what the data reveals, particularly around the labelled bearing fault period

Question 2: Modelling

Using the labelled data, build a machine learning model that detects signs of bearing fault.

Submit your work as a Jupyter notebook (.ipynb) that clearly shows:

- Feature engineering and selection (you may use or extend the provided utility functions)
- Model selection and justification
- Training procedure
- Evaluation results with appropriate metrics, and your interpretation of them

You are free to frame this as binary classification, anomaly detection, or another approach — justify your choice.

Question 3: System Architecture

Design a production architecture for deploying the model you built in Question 2. Assume the following scenario: 50 centrifuge machines, each equipped with a vibration sensor streaming data at the sampling rate observed in the dataset.

Produce an architecture diagram and a written explanation (1–2 pages) covering:

- **Data Ingestion & Storage** — How sensor data flows from machine to model, and how raw and processed data is stored
- **ML Lifecycle** — How the model is trained, versioned, evaluated, and retrained over time as new labelled data becomes available
- **Inference Strategy** — Whether predictions are made in real-time, near-real-time, or batch, and why
- **Alerting & Integration** — How predictions reach the maintenance team

(dashboards, alerts, ticketing systems, etc.)
- **Scalability** — How the architecture handles growth to 500+ machines

You may use any cloud provider or open-source tooling. Justify your key technology choices.